Data Science (COL2010) — Homework 3

Module 3: Clustering

Total: 100 marks Part I: 40 Part II: 60

What you may use: a small Python/NumPy reference

You are **not** expected to search through the entire NumPy/SciPy/Pandas API. The functions below are sufficient for the reference solution. You may use these functions unless a task explicitly states otherwise.

Important. Library restriction. You may use the Python standard library and NumPy. **scikit-learn (`sklearn`) is not allowed anywhere in this homework.** SciPy and pandas are not required. Do not replace a requested implementation with an existing clustering, mixture-model, PCA, or clustering-validation routine. Matplotlib is used only inside the supplied visualization helpers generated by `setup.py`; plotting itself is not assessed.

Library/function	What it is useful for in this homework
`numpy as np`	Numerical arrays, vectorized arithmetic, linear algebra, and random sampling.
`np.asarray`, `np.array`	Convert data to NumPy arrays or construct arrays explicitly.
`np.zeros`, `np.ones`, `np.full`, `np.eye`	Create initialized arrays, labels, counters, and identity matrices.
`np.arange`	Produce integer indices such as $0, \ldots, N-1$.
`np.sum`, `np.mean`, `np.min`, `np.max`	Aggregate distances, cluster coordinates, CF statistics, and scores.
`np.argmin`, `np.argmax`	Locate the nearest centroid or largest assignment.
`np.any`	Test whether any array entry satisfies a Boolean condition.
`np.unique`, `np.bincount`	Find distinct cluster labels and count cluster membership.
`np.where`, `np.argwhere`	Obtain indices at which a Boolean condition is true.
`np.column_stack`	Assemble several one-dimensional arrays as matrix columns.
`np.vstack`	Assemble several one-dimensional arrays as matrix rows.
`np.setdiff1d`	Find elements not already selected, useful for robust K-means++ initialization.
`np.sqrt`, `np.exp`, `np.log`	Euclidean distance and information-theoretic validation calculations.
`np.array_equal`	Check whether two cluster-assignment arrays are identical.
`np.linalg.norm`	Measure centroid movement, including the Frobenius norm of a centroid matrix.
`np.linalg.svd`	Singular-value decomposition used to compute PCA coordinates.
`A @ B`	Matrix multiplication, used in PCA and vector calculations.
Boolean indexing, e.g. `X[labels == k]`	Select all observations currently assigned to cluster k.
Broadcasting, e.g. `X[:,None,:]-C[None,:,:]`	Compute many point–centroid or point–point differences without explicit nested loops.
`np.random.default_rng(seed)`	Create a reproducible random-number generator.
`rng.integers`, `rng.choice`	Random initialization, weighted K-means++ sampling, and mini-batch sampling.
`collections.deque`	Efficient FIFO queue for DBSCAN cluster expansion.
Python `set`	Fast membership checking, used to avoid adding the same DBSCAN neighbour repeatedly.

Files, data, and visualization

1. Run `python setup.py`. It creates `module3_data.npz`, downloads/caches the real datasets, and generates `visualization.py`.

2. The first run needs an Internet connection for the real datasets. Downloaded files are cached in `module3_real_data/`.
3. Submit only your completed `solutions.py`. Do not alter function signatures or required output labels.
4. The autograder calls functions directly. Return values and printed labels are both checked.
5. Ground-truth labels supplied with synthetic/real datasets are for **evaluation only**; they must never be used while fitting a clustering algorithm.

How parameters are supplied and how to test your functions

For Tasks 1–24, the supplied `skeleton-solutions.py` contains a corresponding helper `get_taskN_inputs()`. The helper returns the example arguments for that task in the same order as the function interface. For example:

```
X, K, seed = get_task5_inputs()
C = initialize_kmeans_plus_plus(X, K, seed)
```

and

```
X, eps, min_pts = get_task22_inputs()
labels = custom_dbscan(X, eps, min_pts)
```

You do **not** need to open or inspect `module3_data.npz` directly. Do not modify the `get_taskN_inputs()` helpers.

The supplied values are only convenient local test cases. The autograder may call your function with **different valid values** of every parameter. Therefore, do not hard-code values such as `K=3`, `eps=0.20`, or a particular data shape merely because they occur in a supplied example.

You are also encouraged to construct small inputs of your own. For example:

```
X = np.array([[0.0, 0.0], [0.1, 0.2], [5.0, 5.0], [5.2, 4.9]])
labels, C, history = custom_kmeans_clustering(X, K=2)
```

For Task 24, datasets are obtained by name rather than through one large task-input function:

```
available_datasets()
X, y = load_dataset("synth_moons")
X, y = load_dataset("iris")
```

Here `X` is the feature matrix used by the clustering algorithm and `y` is supplied only for external evaluation. The older names `get_laboratory_dataset_names()` and `get_laboratory_dataset(name)` are retained as aliases.

Supplied visualization code

Plotting is not the learning objective. The generated `visualization.py` supplies:

```
plot_clusters(X_2d, labels, title="Clustering")
compare_clusterings(X_2d, label_sets, title=None)
show_digit_examples(X, true_labels=None, cluster_labels=None, n=12)
compute_tsne_projection(X, perplexity=30.0, n_iters=350, seed=42)
```

For a two-dimensional synthetic dataset, pass its original coordinates directly. For a dataset with more than two features, cluster in the full feature space (or in the requested PCA-reduced feature space) and use `compute_pca_projection(X, 2)` for common PCA plotting coordinates. The supplied `compute_tsne_projection` helper is for exploratory visualization only; do not use its coordinates as input to a clustering algorithm. Every panel in a direct algorithm comparison must use the same coordinates; only the colours/labels change.

Clustering laboratory datasets

Name	Type	Purpose
`synth_blobs`	Synthetic, 2D	Compact spherical clusters: most methods should broadly agree.
`synth_aniso`	Synthetic, 2D	Elongated/rotated clusters: exposes the spherical-centroid assumption of K-means.
`synth_unequal`	Synthetic, 2D	Unequal variances/densities: exposes assumptions behind centroid and density methods.
`synth_moons`	Synthetic, 2D	Two non-convex moons: useful for contrasting centroid, linkage, and density methods.
`synth_circles`	Synthetic, 2D	Concentric rings: a particularly clear failure case for centroid geometry.
`synth_noise`	Synthetic, 2D	Compact groups plus noise: DBSCAN can leave points unassigned.
`synth_highdim`	Synthetic, 50D	Five informative coordinates plus 45 nuisance coordinates: study distance concentration and PCA.
`iris`	Real, 4D	Three iris species; standard low-dimensional benchmark.
`wine`	Real, 13D	Three cultivars; feature scales differ strongly, so the provided matrix is standardized.
`penguins`	Real, 4D	Palmer penguin body measurements and species labels.
`wholesale`	Real, 6D	Customer spending data; channel metadata is an external reference, not necessarily a "true clustering."
`digits`	Real, 64D	UCI handwritten digits 0–4; high-dimensional pixel vectors for PCA/t-SNE visualization and clustering experiments.
`scale_500`	Synthetic, 2D	500 observations for the scalability comparison.
`scale_2000`	Synthetic, 2D	2,000 observations for the scalability comparison.
`scale_6000`	Synthetic, 2D	6,000 observations for the scalability comparison.

The real datasets are downloaded by `setup.py` from the UCI Machine Learning Repository (Iris, Wine, Wholesale Customers, Optical Recognition of Handwritten Digits) and the Palmer Penguins project. No `sklearn.datasets` calls are used.

Learning outcomes

After completing this homework, you should be able to formulate K-means as coordinate descent, implement K-means++ and mini-batch updates, manipulate linkage criteria using Lance–Williams updates, interpret dendrogram cuts, distinguish internal and external validation, identify DBSCAN core/border/noise points, compute and use PCA projections, explain the role and limitations of t-SNE visualization, construct and merge BIRCH clustering features, implement a simplified BIRCH-style summarization procedure, and explain why different clustering algorithms behave differently on different geometries, dimensions, and data scales.

Part I: Foundational components (40 marks)

Task 1: Squared Euclidean distance **[2 marks]**

Purpose: K-means geometry

Reference: PRML §9.1

Compute $\|x_q - x_r\|_2^2$.
Supplied test input:

```
x_q, x_r = get_task1_inputs()
```

Strict Python interface

```
def calculate_squared_euclidean_distance(x_q: np.ndarray, x_r: np.ndarray) -> float:
```

Exploration. Using `synth_highdim`, compare nearest and farthest distances as dimensions are progressively added. Observe the concept of "distance concentration": here.

Task 2: Assignment indicator matrix [2 marks]

Purpose: K-means hard assignments

Reference: PRML Eqs. 9.1–9.2

Return a one-hot matrix $R \in \{0,1\}^{N \times K}$ assigning each point to its nearest centroid. Note that in this part, you only need to get the cluster assignments for cluster centroids `C` and data points `X`.
Supplied test input:

```
X, C = get_task2_inputs()
```

Strict Python interface

```
def build_assignment_indicator_matrix(X: np.ndarray, C: np.ndarray) -> np.ndarray:
```

Exploration. Use the supplied plotting helper to visualize the 2D assignments induced by three different centroid placements.

Task 3: Centroid update [2 marks]

Purpose: K-means centroid update

Reference: PRML Eq. 9.4

Compute each centroid as the mean of assigned points; use the global mean for an empty cluster.
Supplied test input:

```
X, assignments, K = get_task3_inputs()
```

Strict Python interface

```
def update_cluster_centroids(X: np.ndarray, assignments: np.ndarray, K: int) -> np.ndarray:
```

Exploration. Compare three empty-cluster policies: global mean, farthest point, and keeping the old centroid.

Task 4: K-means distortion [2 marks]

Purpose: K-means objective

Reference: PRML Eq. 9.1

Compute $J = \sum_n \|x_n - c_{z_n}\|_2^2$.
Supplied test input:

```
X, C, labels = get_task4_inputs()
```

Strict Python interface

```
def compute_kmeans_distortion(X: np.ndarray, C: np.ndarray, labels: np.ndarray) -> float:
```

Exploration. Verify empirically that one assignment step followed by one centroid-update step does not increase J.

Task 5: K-means++ initialization [2 marks]

Purpose: K-means++ initialization

Reference: Arthur and Vassilvitskii

Choose the first centroid uniformly, then sample later centroids proportional to squared distance from the nearest already-chosen centroid.
Supplied test input:

```
X, K, seed = get_task5_inputs()
```

Strict Python interface

```
def initialize_kmeans_plus_plus(X: np.ndarray, K: int, seed: int = 42) -> np.ndarray:
```

FAQ: What is the seed there for? **Ans:** Learn about `np.random.default_rng` here. You can pass the `seed` parameter to this constructor and use the random number generator (rng) to generate the first index chosen as the cluster centre using rng.integers(). (Read more about it: here). Moreover, you can use rng.choice() to choose one of the indices according to a known distribution. (Read more about it: here).

Exploration. Notice that you can use the functions that you have made to write a complete K-means clustering algorithm (See Task 19). When does K-means converge? Compare the number of steps taken by K-means algorithm to converge under K-means++ and random initialization.

Task 6: Mini-batch centroid update [2 marks]

Purpose: Mini-batch K-means

Reference: Sculley, WWW 2010

Process the mini-batch sequentially. If x is assigned to centroid k, increment v_k and update $c_k \leftarrow c_k + (x - c_k)/v_k$.

Supplied test input:

```
X, C, batch_indices, counts = get_task6_inputs()
```

Strict Python interface

```
def mini_batch_centroid_update(X: np.ndarray, C: np.ndarray,
                               batch_indices: np.ndarray, counts: np.ndarray) -> tuple:
```

Exploration. Plot distortion versus number of processed mini-batches for several batch sizes.

Task 7: Single linkage [2 marks]

Purpose: Hierarchical clustering

Reference: IDM hierarchical clustering

Return the minimum pairwise distance between clusters a and b.

Supplied test input:

```
X, labels, a, b = get_task7_inputs()
```

For Tasks 7-9, `X` is a numpy array of data points of size $n \times d$, `labels` is a numpy array of labels of size n. `a` and `b` are the clusters between which the linkage has to be computed. It is guaranteed that the clusters a and b will be non-empty.

Strict Python interface

```
def compute_single_linkage_distance(X: np.ndarray, labels: np.ndarray, a: int, b: int) -> float:
```

Exploration. Visualize the chaining effect using a bridge of points between two groups.

Task 8: Complete linkage [2 marks]

Purpose: Hierarchical clustering

Reference: IDM hierarchical clustering

Return the maximum pairwise distance between two clusters.
Supplied test input:

```
X, labels, a, b = get_task8_inputs()
```

Strict Python interface

```
def compute_complete_linkage_distance(X: np.ndarray, labels: np.ndarray, a: int, b: int) -> float:
```

Exploration. Compare cluster compactness under single and complete linkage.

Task 9: Average linkage [2 marks]

Purpose: Hierarchical clustering

Reference: ESL §14.3.12

Return the mean of all pairwise distances across the two clusters.
Supplied test input:

```
X, labels, a, b = get_task9_inputs()
```

Strict Python interface

```
def compute_average_linkage_distance(X: np.ndarray, labels: np.ndarray, a: int, b: int) -> float:
```

Exploration. Compare all three linkage choices on one synthetic dataset using the supplied comparison plot.

Task 10: Lance–Williams recurrence [2 marks]

Purpose: Hierarchical proximity updates

Reference: Lance–Williams recurrence

Given distances between three distinct, non-empty clusters A, B, C and their positive integer sizes n_a, n_b, n_c, return the updated dissimilarity from $A \cup B$ to C:

$$d_{(A\cup B),C} = \alpha_a d_{ac} + \alpha_b d_{bc} + \beta d_{ab} + \gamma |d_{ac} - d_{bc}|.$$

Implement all four linkage choices using these coefficients, with $T = n_a + n_b + n_c$:

Linkage	α_a	α_b	β	γ
`single`	$1/2$	$1/2$	0	$-1/2$
`complete`	$1/2$	$1/2$	0	$1/2$
`average`	$n_a/(n_a+n_b)$	$n_b/(n_a+n_b)$	0	0
`ward`	$(n_a+n_c)/T$	$(n_b+n_c)/T$	$-n_c/T$	0

For single, complete, and average linkage, the inputs are the corresponding ordinary linkage distances from the slides. For `ward`, this task uses **squared Ward dissimilarities**:

$$d_{AB} = \frac{2n_a n_b}{n_a + n_b}\|\mu_A - \mu_B\|_2^2.$$

Thus the Ward input and output are on the squared scale; do not square the supplied values again or take a square root of the updated value. Return the updated dissimilarity as a Python float. The Ward row is supplementary material supplied here. Unsupported linkage names must raise `ValueError`.

Supplied test input:

```
d_ac, d_bc, d_ab, n_a, n_b, n_c, linkage = get_task10_inputs()
```

Strict Python interface

```
def lance_williams_update(d_ac: float, d_bc: float, d_ab: float,
                          n_a: int, n_b: int, n_c: int, linkage: str) -> float:
```

Exploration. Numerically verify the single, complete, and average updates against distances recomputed directly from points.

Task 11: Dendrogram cut [2 marks]

Purpose: Dendrogram interpretation

Reference: IDM hierarchical clustering

The one-dimensional, sorted, non-negative `merge_distances` contains the complete $N-1$ merge heights of a binary agglomerative hierarchy. Starting with $N = \text{len}(\texttt{merge_distances}) + 1$ singleton clusters, include every merge whose height is **less than or equal to** the finite `cut_height`. Return the resulting Python integer number of clusters. Tied merge heights are all included when they equal the cut. An empty merge array represents one observation and returns 1.

Supplied test input:

```
merge_distances, cut_height = get_task11_inputs()
```

Strict Python interface

```
def count_clusters_at_dendrogram_cut(merge_distances: np.ndarray,
                                     cut_height: float) -> int:
```

Exploration. Plot merge height versus merge number and show how two different cuts alter the number of clusters.

Task 12: Mean silhouette score [3 marks]

Purpose: Clustering validation

Reference: IDM; ESL

For finite numerical X of shape (N, D) with $N, D \geq 1$ and a one-dimensional array of N cluster labels, implement the mean silhouette coefficient using Euclidean distances. For a point in a non-singleton cluster, let $a(i)$ be its mean distance to the *other* points in that cluster and $b(i)$ be the smallest mean distance to any other cluster. Use

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \qquad S = \frac{1}{N} \sum_i s(i).$$

Set $s(i) = 0$ for a singleton cluster or when both $a(i)$ and $b(i)$ are zero. For this homework, define $S = 0$ when there is only one cluster; consequently, a partition consisting entirely of singleton clusters also has score zero. Treat every distinct label, including `-1`, as an ordinary cluster label; do not drop noise-labelled rows. Return a Python float and do not modify either input.

Supplied test input:

```
X, labels = get_task12_inputs()
```

Strict Python interface

```
def compute_mean_silhouette_score(X: np.ndarray, labels: np.ndarray) -> float:
```

Exploration. Compare silhouette scores for several values of K on `synth_blobs`.

Task 13: Cluster purity [3 marks]

Purpose: Clustering validation

Reference: IDM clustering validation

Given a two-dimensional contingency matrix of non-negative integer counts, use **rows for discovered clusters** and **columns for true classes**, as in the slides. The matrix may be rectangular. With $N = \sum_{k,j} n_{kj}$, return the Python float

$$\text{purity} = \frac{\sum_k \max_j n_{kj}}{N}.$$

If the total count is zero, return 0.0. Zero-count rows and columns are allowed. Do not modify the matrix. The supplied example has different row- and column-based maxima sums, so the orientation matters.

Supplied test input:

```
contingency = get_task13_inputs()
```

Strict Python interface

```
def compute_cluster_purity(contingency: np.ndarray) -> float:
```

Exploration. Construct a case showing why purity can increase when a clustering is split into many small clusters.

Task 14: Normalized mutual information [3 marks]

Purpose: Clustering validation

Reference: IDM; ESL §14.3

Use the same contingency orientation and valid counts as Task 13. For positive total count N, set $p_{kj} = n_{kj}/N$, $p_k = \sum_j p_{kj}$, and $p_j = \sum_k p_{kj}$. Compute

$$I(C;Y) = \sum_{k,j:p_{kj}>0} p_{kj} \log \frac{p_{kj}}{p_k p_j}, \quad H(C) = -\sum_{k:p_k>0} p_k \log p_k, \quad H(Y) = -\sum_{j:p_j>0} p_j \log p_j.$$

When both entropies are positive, return $I(C;Y)/\sqrt{H(C)H(Y)}$ using natural logarithms. Omit zero-probability terms. For this homework, return 0.0 for total count zero; for positive total count, return 1.0 if both entropies are zero (both partitions have one occupied group), and 0.0 if exactly one entropy is zero. Return a Python float and do not modify the matrix. Small roundoff excursions outside $[0, 1]$ may be clipped.

Supplied test input:

```
contingency = get_task14_inputs()
```

Strict Python interface

```
def compute_normalized_mutual_information(contingency: np.ndarray) -> float:
```

Exploration. Compare NMI and purity after splitting one true class across several predicted clusters.

Task 15: Rand index [3 marks]

Purpose: Clustering validation

Reference: External clustering validation

Given two labelings of the same N observations, compute the Rand index by considering all unordered pairs. A pair is an agreement if the two labelings either place both observations in the same cluster or place them in different clusters:

$$RI = \frac{TP + TN}{\binom{N}{2}}.$$

The inputs are one-dimensional label arrays of equal length; label names need not match between the arrays. Count each unordered pair once and exclude self-pairs. For $N < 2$, define the Rand index to be 1.0. Return a Python float and do not modify the inputs. The supplied example includes disagreements rather than merely renaming the same partition.

Supplied test input:

```
labels_true, labels_pred = get_task15_inputs()
```

Strict Python interface

```
def compute_rand_index(labels_true: np.ndarray,
                       labels_pred: np.ndarray) -> float:
```

Exploration. Construct two clusterings with the same number of clusters but different pairwise agreements, and compare their Rand indices.

Task 16: DBSCAN point types [2 marks]

Purpose: Density-based clustering

Reference: IDM density-based clustering

For finite numerical X of shape (N, D) with $N, D \geq 1$, non-negative finite `eps`, and positive integer `min_pts`, use Euclidean neighborhoods with distance **less than or equal to** `eps`, counting each point itself. Return an integer NumPy array of shape $(N,)$: 2 for a core point (at least `min_pts` neighbors), 1 for a non-core point lying in the neighborhood of at least one core point, and 0 otherwise. A non-core neighbor of a border point alone is not sufficient to make a point border. Do not modify X.

Supplied test input:

```
X, eps, min_pts = get_task16_inputs()
```

The supplied parameters exercise all three point types.

Strict Python interface

```
def classify_dbscan_points(X: np.ndarray, eps: float,
                           min_pts: int) -> np.ndarray:
```

Exploration. Plot the number of core, border, and noise points as ϵ varies.

Task 17: PCA projection [2 marks]

Purpose: PCA for high-dimensional data

Reference: PRML §12.1; ESL §14.5

For finite numerical X of shape (N, D) with $N, D \geq 1$, center each feature and use reduced SVD $X_c = USV^T$. Let V_k contain the first $k =$ `n_components` columns of V, in decreasing singular-value order, and return the principal-component coordinates $Z = X_c V_k$. Do not standardize the features. The returned float NumPy array must have shape (N, k) and the input must remain unchanged. Require an integer $1 \leq k \leq \min(N, D)$; invalid component counts must raise `ValueError`.

Return the projection array Z; for constant data, its coordinates are zero. Column sign flips and orthonormal basis choices within repeated-eigenvalue subspaces are mathematically equivalent and must be accepted when checking projections.

Supplied test input:

```
X, n_components = get_task17_inputs()
```

Strict Python interface

```
def compute_pca_projection(X: np.ndarray, n_components: int = 2) -> np.ndarray:
```

Exploration. Cluster in the full feature space, plot in 2D PCA coordinates, and explain why visual separation in the projection does not prove separation in the original space.

Part II: Complete algorithms and clustering laboratory (60 marks)

Task 18: BIRCH clustering-feature statistics [2 marks]

Purpose: Scalable clustering

Reference: BIRCH CF vector

For finite numerical X of shape (N, D) with $N, D \geq 1$, compute the BIRCH clustering feature

$$CF = (N, LS, SS), \qquad LS = \sum_{i=1}^{N} x_i, \qquad SS = \sum_{i=1}^{N} \|x_i\|_2^2.$$

Compute the centroid and root-mean-square distance to the centroid:

$$\mu = \frac{LS}{N}, \qquad R = \sqrt{\frac{1}{N}\sum_i \|x_i - \mu\|_2^2} = \sqrt{\frac{SS}{N} - \|\mu\|_2^2}.$$

The two radius formulas are equal in exact arithmetic. You may compute the radius from centered differences to avoid catastrophic cancellation for large coordinate offsets. Small negative values caused only by floating-point roundoff in the CF formula may be clipped to zero before the square root; clipping alone does not recover variance already lost through cancellation.

Return `(N, LS, SS, centroid, radius)`, with N a Python integer, `LS` and `centroid` float NumPy arrays of shape $(D,)$, and `SS` and `radius` Python floats. In particular, `SS` is a scalar sum of squared norms, not a featurewise vector. A singleton has radius zero. Do not modify X.

Supplied test input:

```
X = get_task18_inputs()
```

Strict Python interface

```
def compute_birch_cf_statistics(X: np.ndarray) -> tuple:
```

Exploration. Split one dataset into two subsets, compute a CF for each subset, add the two CFs, and verify that the merged CF gives the same centroid and radius as computing the CF directly from the union.

Task 19: Custom K-means engine [10 marks]

Purpose: K-means

Reference: PRML §9.1

Implement Lloyd's algorithm with `random` and `kmeans++` initialization (string passed as argument), nearest-centroid assignment, centroid updates, empty-cluster handling, and distortion history.
Return `(labels, centroids, distortion_history)`. **Supplied test input:** `X, K = get_task19_inputs()`

Strict Python interface

```
def custom_kmeans_clustering(X: np.ndarray, K: int, max_iters: int = 100,
                             seed: int = 42, init: str = 'kmeans++') -> tuple:
```

Exploration. Compare random and K-means++ initializations over at least 30 seeds.

Task 20: Mini-batch K-means [10 marks]

Purpose: K-means – scalable clustering

Reference: Sculley, WWW 2010

Implement stochastic mini-batch centroid updates using per-centroid counts of number of data points assigned. Return `(labels, centroids, distortion_history, counts)`. **Supplied test input:** `X, K, batch_size = get_task20_inputs()`

Strict Python interface

```
def custom_minibatch_kmeans(X: np.ndarray, K: int, batch_size: int,
                            max_iters: int = 200, seed: int = 42) -> tuple:
```

Exploration. Compare runtime and final distortion with full-batch K-means as N increases.

Task 21: Agglomerative clustering with merge history [10 marks]

Purpose: Hierarchical clustering

Reference: IDM hierarchical clustering

Start with singleton clusters, repeatedly merge the closest pair, support `single`, `complete`, and `average` linkages, and record `(cluster_a, cluster_b, distance, merged_size)`. Reuse the proximity-update ideas from Task 10 rather than repeatedly invoking a library routine. Return `(labels, merge_history)`. **Supplied test input:** `X, K, linkage = get_task21_inputs()`

Strict Python interface

```
def custom_agglomerative_clustering(X: np.ndarray, K: int,
                                    linkage: str = 'average') -> tuple:
```

Exploration. Compare the three linkage choices on `synth_moons` and `synth_noise`.

Task 22: Custom DBSCAN [10 marks]

Purpose: Density-based clustering

Reference: IDM density-based clustering

Implement density reachability and cluster expansion from scratch. Noise must be labelled -1. A point initially marked noise must be reassigned if later reached from a core point. Return `labels`. **Supplied test input:** `X, eps, min_pts = get_task22_inputs()`

Strict Python interface

```
def custom_dbscan(X: np.ndarray, eps: float, min_pts: int) -> np.ndarray:
```

Exploration. Compare DBSCAN and K-means visually on the moon and noise datasets while varying ϵ and `min_pts`.

Task 23: Simplified BIRCH clustering [10 marks]

Purpose: Scalable clustering

Reference: BIRCH clustering features and CF summarisation

Implement an educational BIRCH-style clustering procedure using the clustering-feature statistics from Task 18. You are **not** required to implement disk pages or the full height-balanced CF-tree split machinery.

Process the observations once, in input order, maintaining a list of leaf CF entries:

1. Represent a new point by its one-point CF.
2. If no leaf entry exists, create one.
3. Otherwise find the existing leaf CF whose centroid is nearest to the new point.
4. Tentatively merge the point with that CF using additivity of (N, LS, SS).
5. Absorb the point if the merged radius is at most `threshold`; otherwise create a new leaf CF.

After the scan, globally cluster the resulting leaf centroids into K groups using your Task 19 K-means implementation. Finally assign every original observation the global cluster label of its nearest leaf centroid.

Return `(labels, cf_counts, cf_centroids, cf_radii)`.

Supplied test input: `X, K, threshold = get_task23_inputs()`

Strict Python interface

```
def custom_birch_clustering(X: np.ndarray, K: int, threshold: float,
                            seed: int = 42) -> tuple:
```

Exploration. Vary `threshold`. Explain the trade-off between a small threshold (many fine-grained CF summaries) and a large threshold (stronger compression but possible loss of cluster detail).

Task 24: Clustering laboratory [10 marks]

Purpose: Clustering algorithms, dimensionality reduction, scalability, and validation

Reference: Module synthesis

This task is about **algorithmic assumptions and computational trade-offs**, not simply finding the largest score. Implement the wrapper below by calling your own Tasks 19–23. It must run K-means, mini-batch K-means, average-link agglomerative clustering, DBSCAN, and simplified BIRCH; return each algorithm's labels and relevant diagnostics; and compute silhouette, NMI, and Rand index for each labeling.

Supplied wrapper test input: `X, labels_true, K, dbscan_eps, dbscan_min_pts, birch_threshold = get_task24_inputs()`

Laboratory dataset access: `X, y = load_dataset("synth_moons")` (and analogously for the other named datasets).

Strict Python interface

```
def run_clustering_laboratory(X: np.ndarray, labels_true: np.ndarray, K: int,
                              dbscan_eps: float, dbscan_min_pts: int,
                              birch_threshold: float = 0.75,
                              seed: int = 42) -> dict:
```

Then use `available_datasets()` and `load_dataset(name)` together with the supplied visualization functions.

A. Geometry laboratory. Use all six 2D synthetic datasets and at least three of `iris`, `wine`, `penguins`, and `wholesale`. For every dataset:

1. run K-means, mini-batch K-means, agglomerative clustering, DBSCAN, and simplified BIRCH;
2. plot all labelings on the *same* coordinates (original 2D coordinates for synthetic data, 2D PCA coordinates for real data);

3. report silhouette, NMI, and Rand index;
4. identify at least one important agreement or disagreement among algorithms and explain it using their geometric, density, hierarchy, or compression assumptions.

For DBSCAN and BIRCH, you may try a small number of reasonable `eps`, `min_pts`, and `threshold` values. Do not perform an exhaustive library/grid search.

B. High-dimensional laboratory: PCA and t-SNE. Use both `synth_highdim` (50D) and `digits` (64D). For each:

1. compare a nearest-distance and farthest-distance summary in the full space;
2. run K-means in the full space;
3. repeat K-means after PCA to 2, 10, and 20 dimensions (where available);
4. report Rand index, NMI, and silhouette for each clustering representation;
5. produce both a 2D PCA plot and a supplied t-SNE plot using `compute_tsne_projection`;
6. **do not cluster in the t-SNE coordinates**; use t-SNE only to inspect local neighbourhood structure;
7. explain whether PCA helps by removing nuisance/redundant dimensions, and explain why distances between separated t-SNE islands should not be overinterpreted.

You are **not required** to run the from-scratch agglomerative implementation on the full high-dimensional datasets; explain the computational reason using its pairwise-distance costs.

C. Scalability laboratory. Use `scale_500`, `scale_2000`, and `scale_6000`. On each dataset:

1. run full K-means, mini-batch K-means, and simplified BIRCH using the same target K;
2. measure wall-clock runtime using Python's `time.perf_counter`;
3. report final distortion for K-means and mini-batch K-means, and the number of BIRCH leaf CFs;
4. compare how runtime and compression change as N grows;
5. explain why mini-batches and CF summaries are useful when the full dataset becomes large.

Important. A supplied external label is not automatically the "correct clustering." In particular, real-world metadata can encode a categorization that does not coincide with the strongest geometric structure in the chosen features. Use external validation scores as comparisons to supplied labels, not as the sole definition of success.

Submission checklist

Run `python setup.py`; ensure every signature matches the skeleton; preserve required printed labels; do not import or call `sklearn`; use the supplied visualization helper rather than spending time implementing plotting; submit only the completed `solutions.py` unless instructed otherwise.